ReconX

Next-Generation Automated Reconnaissance Framework

Technical Documentation

Version 8.0.0

Prepared by

Divyanshu Saini

Department of Computer Science and Engineering

February 4, 2026

Project	ReconX – Automated Reconnaissance Framework
License	MIT License
Repository	`github.com/DivyanshuSaini2112/recony`

Contents

1 Executive Summary

ReconX is an enterprise-grade, command-line reconnaissance automation framework designed to streamline security assessments for penetration testers and security professionals. The framework orchestrates multiple industry-standard security tools into a unified workflow, producing comprehensive reports in multiple formats.

1.1 Core Capabilities

The framework integrates seven essential security scanning modules:

- **Nmap Integration** - Comprehensive port scanning and service version detection
- **Subfinder** - Passive subdomain enumeration using multiple data sources
- **Gobuster & ffuf** - Web directory and content discovery
- **WhatWeb** - Technology stack fingerprinting and identification
- **Nikto** - Web server vulnerability assessment
- **SQLMap** - Automated SQL injection testing
- **URLScan.io** - Comprehensive web intelligence gathering

1.2 Target Audience

This documentation is designed for security professionals including penetration testers conducting authorized assessments, security researchers performing vulnerability analysis, system administrators evaluating network security posture, and students learning offensive security methodologies.

2 System Architecture

ReconX implements a layered architecture with clear separation of concerns, enabling maintainability and extensibility. The framework consists of four distinct layers working together to provide comprehensive security reconnaissance.

2.1 Architecture Layers

1. **Presentation Layer** - Command-line interface built with argparse for user interaction and parameter specification
2. **Orchestration Layer** - Manages module coordination, concurrency through ThreadPoolExecutor, and workflow management
3. **Module Layer** - Individual tool wrappers with consistent interfaces for each security tool
4. **Utility Layer** - Handles configuration management, output generation, and logging

2.2 Module Interface Contract

Each security tool is wrapped in a dedicated Python module following a consistent contract. This standardization ensures predictable integration and simplified maintenance. All modules implement:

- **Initialization** - Accept target and configuration parameters
- **Command Generation** - Provide command preview for logging and dry-run modes
- **Execution** - Run scans and return normalized results or error dictionaries
- **Parsing** - Convert tool output into standardized Python data structures

2.3 Pipeline Architecture

The framework implements a six-stage pipeline for processing reconnaissance tasks:

1. **Input Validation** - Verify target specification and validate parameters
2. **Planning** - Select modules based on user input and calculate timeouts
3. **Execution** - Run modules in parallel with real-time progress tracking
4. **Collection** - Aggregate results with comprehensive error handling
5. **Processing** - Normalize and enrich collected data
6. **Output** - Generate reports in JSON, text, and HTML formats

This pipeline ensures consistent processing while maintaining resilience against individual tool failures through defensive programming practices.

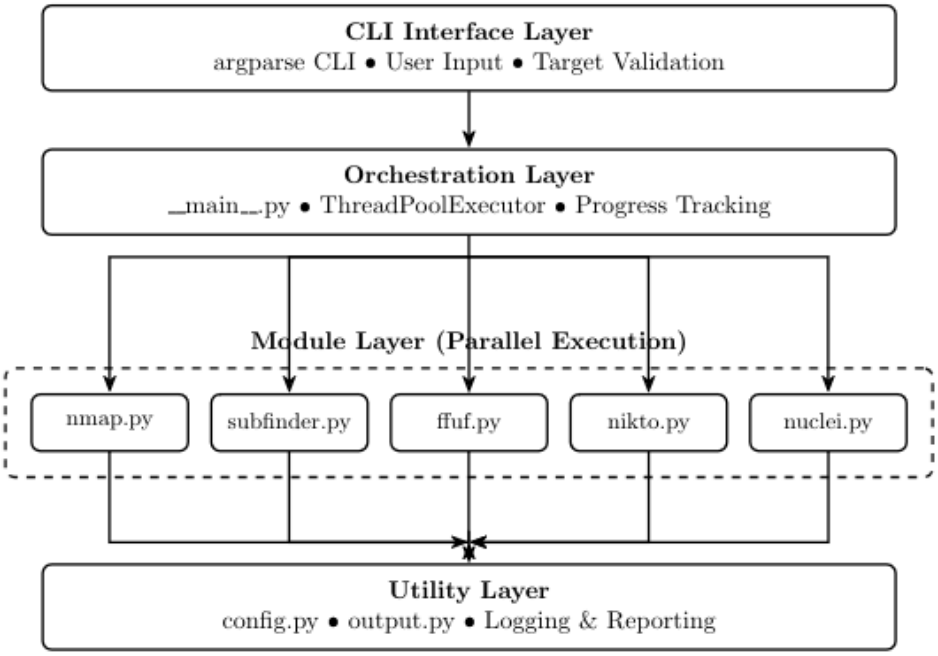

Figure 1: System Architecture Diagram of ReconX

3 Module Specifications

This section describes the integrated security tools and their specific implementations within the ReconX framework.

3.1 Nmap Module

The Nmap module provides comprehensive network reconnaissance through a two-stage scanning process designed for efficiency and thoroughness.

Scanning Stages:

1. **Quick Discovery** - Fast scan to identify open ports using aggressive timing
2. **Detailed Enumeration** - Service version detection and script scanning on discovered ports

The module returns normalized data structures containing IP addresses, port information with protocol and state, service details including product and version, and script outputs. If no ports are discovered in the quick scan, the detailed scan is skipped to optimize execution time.

Error Handling: The module handles missing binaries, timeout situations with partial result recovery, invalid target specifications, and permission issues with appropriate error messages.

3.2 Web Reconnaissance Modules

Subdomain Enumeration (Subfinder): Leverages passive DNS discovery across multiple data sources to identify subdomains. Returns a simple list of discovered subdomain hostnames without requiring active DNS querying.

Directory Fuzzing (Gobuster & ffuf): Both tools provide directory and file discovery capabilities with unified interfaces. Gobuster offers straightforward enumeration, while ffuf provides JSON output for structured parsing, advanced matching and filtering capabilities, and extensive HTTP request customization. The user can select their preferred fuzzer via command-line arguments.

Technology Detection (WhatWeb): Identifies web technologies, frameworks, and server software using an extensive signature database. Returns raw JSON arrays containing all detected technologies with version information where available.

3.3 Vulnerability Assessment Modules

Nikto Scanner: Performs comprehensive web server vulnerability and misconfiguration detection. The module parses output files to extract finding lines marked with plus signs, indicating discovered vulnerabilities or security concerns.

SQLMap Integration: Automates SQL injection testing with configurable risk and level parameters. Returns structured data indicating vulnerability status, specific vulnerable parameters, and detected database versions. The module supports batch mode operation for unattended scanning.

URLScan.io API: Provides comprehensive web intelligence through API integration. The workflow involves submitting URLs to the API, polling results until scan completion, and parsing response data to extract technologies, IP addresses, associated domains, and network calls. Requires API key configuration in the config file.

4 Configuration and Installation

4.1 System Requirements

Operating System: Kali Linux 2024.x is recommended, with support for Ubuntu 20.04+, Debian 11+, and Parrot Security OS 4.x+.

Python Environment:

- Python 3.8 or higher
- pip package manager
- virtualenv (recommended for isolation)

External Dependencies:

- **Nmap** 7.80+ - Port scanning and service detection
- **Subfinder** 2.5.0+ - Passive subdomain enumeration
- **Gobuster** 3.1.0+ / **ffuf** 1.5.0+ - Directory fuzzing
- **WhatWeb** 0.5.5+ - Web technology identification
- **Nikto** 2.1.6+ - Web server vulnerability scanning
- **SQLMap** 1.5+ - SQL injection testing

4.2 Installation Steps

Step 1: Install System Packages

Update your system and install the required security tools:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y nmap gobuster ffuf whatweb
    nikto sqlmap golang-go git python3-pip
```

Step 2: Install Go-Based Tools

Install Subfinder using Go package manager:

```
go install -v github.com/projectdiscovery/
    subfinder/v2/cmd/subfinder@latest
export PATH=$PATH:$(go env GOPATH)/bin
```

Step 3: Install ReconX

Clone the repository and install using pip:

```
git clone https://github.com/DivyanshuSaini2112/
    recony.git
cd recony
pip install .
```

For isolated environment (recommended):

```
python3 -m venv venv
source venv/bin/activate
pip install .
```

4.3 Configuration File

ReconX uses an INI-format configuration file searched in: current directory (./config.ini), repository root (../config.ini), or user home (/.config/reconx/config.ini).

Configuration Format:

```
[API_KEYS]
URLSCAN_API_KEY = your_api_key_here
GROQ_API_KEY = your_groq_key_here

[SETTINGS]
default_threads = 10
default_wordlist = /usr/share/wordlists/
    dirb/common.txt
```

Verification: Run `reconx -help` to verify successful installation and check tool availability using `which` command for each dependency.

5 Usage and Command Reference

5.1 Command Structure

The basic ReconX command requires a target specification and accepts various optional parameters for customization.

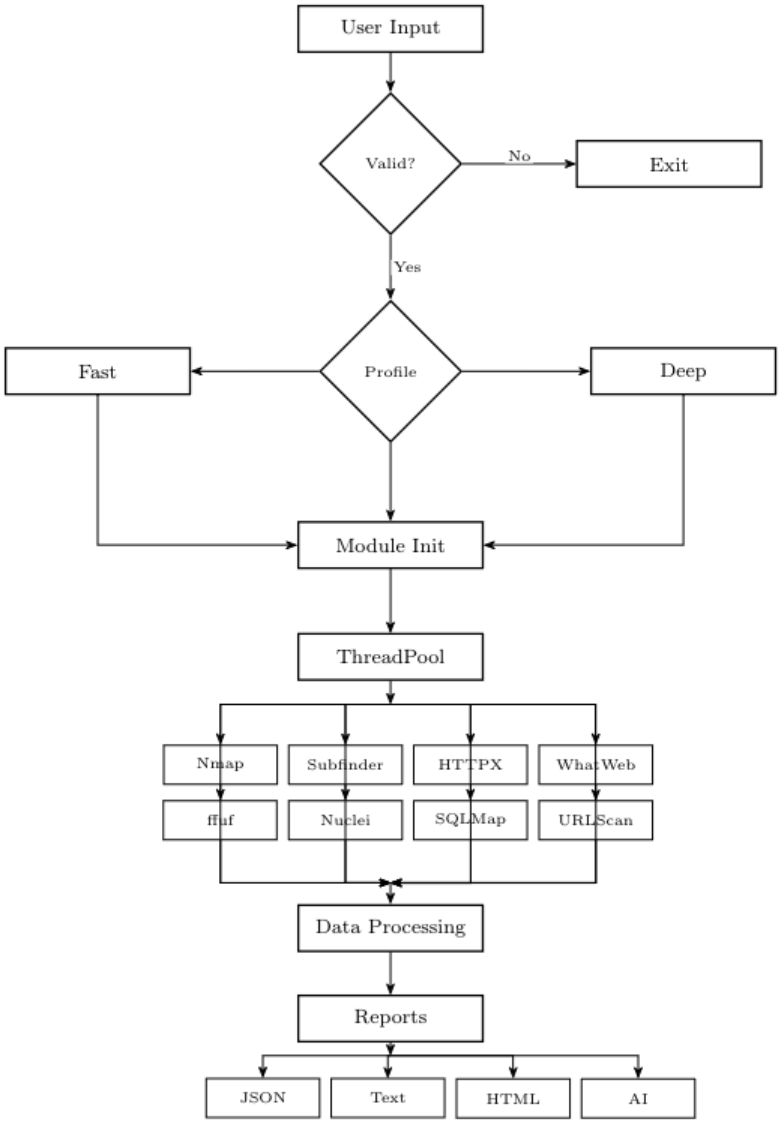

Figure 2: Execution flow diagram of ReconX.

Basic Syntax:

```
reconx -t <target> [options]
```

Key Parameters:

- `-t, -target` - Target IP, domain, or CIDR range (required)
- `-modules` - Comma-separated module list (default: all)
- `-profile` - Scan intensity: fast, default, or deep
- `-threads` - Concurrent thread count (default: 10)
- `-html` - Generate interactive HTML report
- `-no-exec` - Preview mode without execution

5.2 Scan Profiles

ReconX provides three predefined profiles optimized for different use cases:

- **Fast Profile** (5 minutes) - Quick reconnaissance and initial assessment
- **Default Profile** (30 minutes) - Standard comprehensive scanning
- **Deep Profile** (2 hours) - Extensive enumeration and thorough analysis

5.3 Usage Examples

Quick Reconnaissance:

```
reconx -t example.com --profile fast
    --modules nmap,subenum
```

Performs initial target profiling with port scanning and subdomain enumeration, completing in 5-10 minutes.

Standard Security Assessment:

```
reconx -t example.com --profile default
    --modules nmap,subenum,dirfuzz,whatweb,nikto
    --html
```

Conducts comprehensive security evaluation with multiple modules and HTML report generation.

Deep Vulnerability Assessment:

```
reconx -t example.com --profile deep
    --modules all --threads 30
    --wordlist /usr/share/seclists/Discovery/
        Web-Content/raft-large-directories.txt
    --html --ai-summary
```

Performs thorough assessment with all modules, custom wordlists, and AI-powered analysis.

Web Application Focus:

```
reconx -t https://webapp.example.com
    --modules dirfuzz,whatweb,nikto,sqlmap
    --fuzzer ffuf --threads 50
```

Targets web-specific reconnaissance using ffuf for faster directory fuzzing.

Network Range Scanning:

```
reconx -t 192.168.1.0/24 --modules nmap
    --profile fast --out ./network-scan
```

Scans entire network range for open ports and services.

6 Output and Reporting

6.1 Output Directory Structure

Each scan creates a timestamped directory containing multiple output files for different purposes.

Primary Output Files:

- **reconx_results.json** - Aggregated data in structured JSON format
- **scan.log** - Human-readable summary with formatted sections
- **report.html** - Interactive dashboard (optional, with –html flag)
- **ai_summary.txt** - AI-generated analysis (optional, with –ai-summary)

Module-Specific Logs:

- nmap.log, nmap_quick_scan.xml, nmap_detailed_scan.xml
- subdomains.txt, dirfuzz.log, whatweb.log
- nikto.log, sqlmap.log

6.2 JSON Report Format

The primary JSON file contains structured data from all modules with consistent formatting. Each module's output follows its defined data structure:

- **Nmap** - Arrays of host objects with IP addresses and port details
- **Subdomain Enumeration** - String arrays of discovered hostnames
- **Directory Fuzzing** - String arrays of discovered paths
- **WhatWeb** - JSON arrays of detected technologies
- **Nikto** - String arrays of vulnerability findings
- **SQLMap** - Objects with vulnerability status and database details
- **URLScan** - Objects with technologies, IPs, domains, and network calls

6.3 HTML Report Features

The optional HTML report provides visual presentation with multiple sections:

- **Executive Summary** - Risk scoring and key statistics
- **Statistics Cards** - Port count, subdomain count, directory count
- **Findings Tables** - Detailed module results with formatting
- **Risk Assessment** - Color-coded risk levels (LOW/MEDIUM/HIGH/CRITICAL)

Risk Calculation Factors:

- Critical ports discovered (21, 23, 3389, 445, 3306, 5432)
- SQL injection vulnerabilities detected
- Number of exposed directories
- Nikto vulnerability findings

6.4 Error Reporting

Errors are integrated across all output formats for comprehensive visibility. The orchestration layer collects errors from module executions and includes them in console output with warning styling, dedicated error sections in scan.log files, and highlighted error panels in HTML reports. Common error types include binary not found, timeout exceeded, network unreachable, invalid output format, permission denied, and API key missing.

7 Error Handling and Resilience

ReconX implements multi-layered error handling to ensure maximum resilience during security assessments. The framework continues execution even when individual modules fail, ensuring successful results are preserved.

7.1 Module-Level Error Handling

Each scanner implements defensive error handling at multiple execution points:

Error Detection and Response:

- **Tool Not Found** - Returns error dictionary indicating missing binary

- **Timeout Exceeded** - Attempts to parse partial results from temporary files
- **Non-Zero Exit Code** - Returns error with stderr content for diagnosis
- **Unexpected Exceptions** - Captures and returns error details

7.2 Parsing Error Handling

Parsers use defensive programming to handle malformed or incomplete output gracefully. XML parsers wrap operations in try-except blocks and extract valid data from partial documents. Individual element parsing failures are logged but don't prevent processing of other elements. JSON parsers handle missing keys with get methods and default values. Text parsers use line-by-line processing that skips malformed lines while preserving valid data.

7.3 Partial Result Recovery

Several modules implement intelligent partial result recovery when full execution fails:

- **Ffuf/Gobuster** - Reads temporary output files even on timeout
- **SQLMap** - Parses log files to extract findings on early termination
- **Nmap** - Returns quick scan results if detailed scan fails

This ensures that valuable reconnaissance data is captured even when scans are interrupted or time out.

7.4 Error Aggregation

The orchestration layer collects and reports all errors comprehensively. Errors appear in console output with visual styling, dedicated error sections in text summary files, and highlighted error panels in HTML reports. This multi-format error reporting ensures visibility regardless of how users interact with scan results.

8 Testing and Quality Assurance

8.1 Unit Testing Framework

ReconX includes comprehensive test suite in `tests/test_parsers.py` to verify parser functionality across all modules.

Test Coverage Areas:

- **Nmap Parsing** - XML structure, port lists, service detection, scripts
- **Subdomain Parsing** - Plain text line extraction
- **Directory Fuzzing** - Gobuster and ffuf output formats
- **WhatWeb Parsing** - JSON structure validation
- **Nikto Parsing** - Finding line extraction
- **SQLMap Parsing** - Result structure and vulnerability detection

8.2 Running Tests

Execute tests using Python's unittest framework or pytest:

```
# Run all tests
python -m unittest discover -v

# Run specific test class
python -m unittest tests.test_parsers.
```

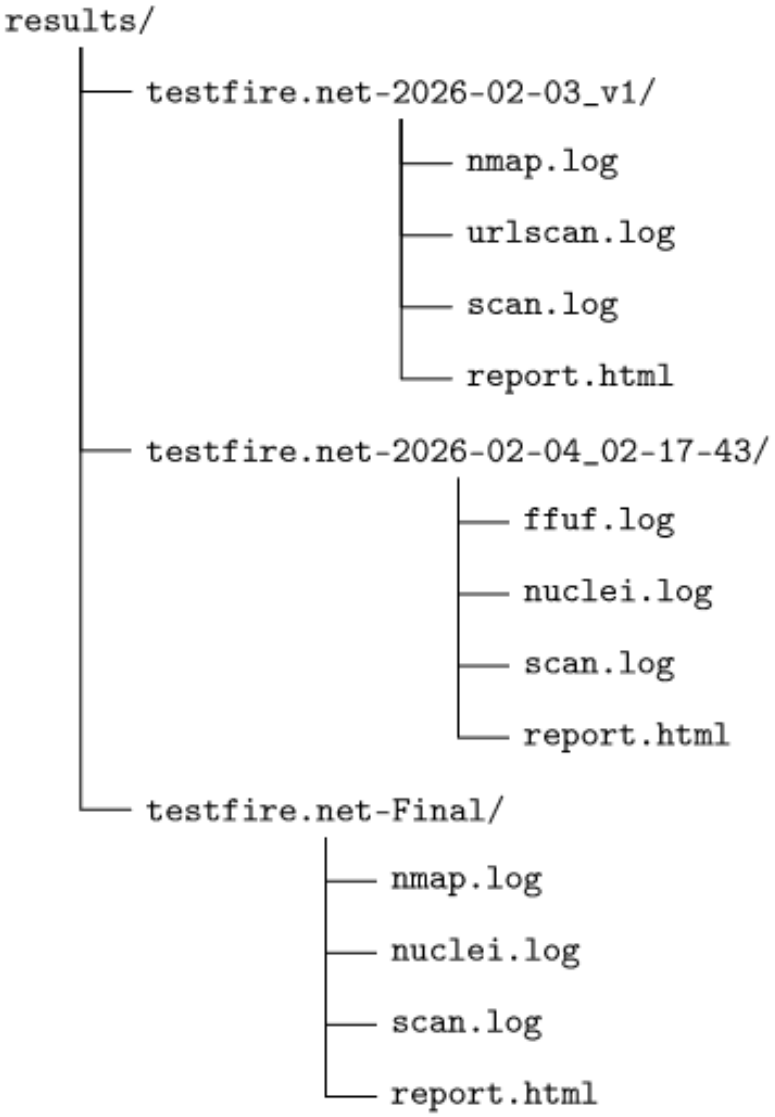

Figure 3: Output directory structure generated by ReconX, showing timestamped scan directories and per-module result files.

```
    TestNmapParser -v

# Run with pytest and coverage
pytest tests/ -v --cov=reconx
coverage html
```

8.3 Integration Testing

For end-to-end testing, use isolated environments and authorized targets:

- Use virtual machines or containers for isolation
- Test against known-safe targets (e.g., scanme.nmap.org)
- Verify complete workflows from input to output
- Validate report generation in all formats

9 Troubleshooting

9.1 Installation Issues

Binary Not Found Error

Symptom: Module returns "command not found" error

Resolution:

- Verify tool installation: `sudo apt install <tool>`
- Check PATH configuration: `which <tool>`
- For Go tools, ensure `$(go env GOPATH)/bin` is in PATH

Permission Denied Error

Symptom: Tools fail with permission errors

Resolution:

- Run with elevated privileges: `sudo reconx ...`
- Configure capabilities: `sudo setcap cap_net_raw+eip $(which nmap)`
- Use unprivileged scan techniques when appropriate

9.2 Execution Issues

Timeout Errors

Symptom: Modules frequently exceed time limits

Resolution:

- Increase profile timeout using `-profile deep`
- Reduce scope with smaller wordlists or fewer threads
- Check network connectivity and target responsiveness

Empty Results

Symptom: Scan completes but returns no findings

Diagnosis Steps:

1. Check module logs in output directory
2. Run tool directly to verify functionality
3. Enable verbose mode with `-v` flag

Common Causes:

- Target host is down or blocking scans
- Firewall/WAF blocking reconnaissance
- Incorrect target specification
- Wordlist path issues

9.3 Performance Optimization

Slow Execution

Optimization Strategies:

- Use `-profile fast` for quick scans
- Increase thread count: `-threads 50`

- Use smaller wordlists for directory fuzzing
- Disable unnecessary modules
- Run scans during off-peak hours

9.4 Debugging Techniques

Useful Debugging Commands:

```
# Enable verbose output
reconx -t example.com -v --modules nmap

# Preview commands without execution
reconx -t example.com --no-exec --modules all

# Check module logs
tail -f results/example.com-*/nmap.log

# Test tools directly
nmap -T4 -F example.com
subfinder -d example.com -silent
```

10 Security and Responsible Use

10.1 Legal and Ethical Obligations

Authorization Requirements:

- Obtain explicit written permission before scanning any target
- Ensure authorization covers all IP ranges and domains
- Understand scope and boundaries of authorization
- Comply with organizational policies and legal jurisdictions

Prohibited Activities:

- Unauthorized network reconnaissance or port scanning
- Scanning systems without explicit permission
- Exceeding authorized scope of assessment
- Using findings for malicious purposes

10.2 Data Protection

ReconX output contains sensitive information that must be protected:

Sensitive Data Types:

- Open ports and service versions (infrastructure details)
- Directory structures (application architecture)
- Subdomain information (organizational structure)
- Potential vulnerabilities (security weaknesses)

Protection Measures:

1. Store output in secure locations with access controls
2. Encrypt sensitive scan results
3. Use secure channels for sharing reports
4. Implement data retention and disposal policies

5. Redact sensitive information when sharing externally

10.3 Operational Security

Scan Detectability: ReconX scans are detectable through network traffic patterns, security monitoring systems (IDS/IPS, WAF), and log file entries.

Mitigation Strategies:

- Coordinate with system administrators for production scans
- Use appropriate scan timing to minimize business impact
- Adjust scan aggressiveness based on target sensitivity
- Be prepared to provide scan source information

10.4 Tool and API Security

Supply Chain Security:

- Verify tool authenticity from official sources
- Keep tools updated to latest stable versions
- Review release notes for security updates
- Use checksums/signatures for critical tools

API Key Best Practices:

- Never commit config.ini to version control
- Use environment variables for API keys in CI/CD
- Rotate API keys regularly
- Limit key permissions to minimum required scope
- Use separate keys for different environments

11 Appendix: Reference Tables

Table 1: Command-Line Arguments Summary

Argument	Description
-t, –target	Target IP, domain, or CIDR range (required)
–modules	Comma-separated module list (default: all)
–profile	Scan intensity: fast, default, deep
–threads	Concurrent thread count (default: 10)
–wordlist	Custom wordlist path
–fuzzer	Directory fuzzer: gobuster or ffuf
–out	Output directory path
–html	Generate HTML report
–ai-summary	Generate AI-powered analysis
–no-exec	Preview mode without execution
-v, –verbose	Enable verbose output

Conclusion

ReconX represents a comprehensive approach to security reconnaissance automation, combining the power of industry-standard tools with a unified, user-friendly interface. The framework's modular architecture with clean separation of concerns enables maintainability and extensibility.

Table 2: Module Return Types

Module	Type	Structure
nmap	List[Dict]	Host objects with IP and ports
subenum	List[str]	Subdomain hostnames
dirfuzz	List[str]	Discovered paths/URLs
whatweb	List[Dict]	WhatWeb JSON objects
nikto	List[str]	Finding strings
sqlmap	Dict	Vulnerability status and details
urlscan	Dict	Technologies, IPs, domains, calls

Table 3: Scan Profile Specifications

Profile	Timeout	Use Case
Fast	5 minutes	Quick reconnaissance, initial assessment
Default	30 minutes	Standard comprehensive scan
Deep	2 hours	Extensive enumeration, thorough analysis

Defensive programming ensures partial results even on tool failures, maintaining operational resilience. Consistent interfaces and comprehensive reporting streamline security assessment workflows. Built-in safeguards and clear documentation promote ethical and legal usage.

The framework is an evolving project that benefits from community feedback and contributions. Security professionals, whether penetration testers, researchers, system administrators, or developers, are encouraged to contribute improvements, report issues, and share use cases. Always obtain proper authorization before conducting security assessments and use ReconX ethically and legally in accordance with all applicable laws and regulations.

For support and contributions, visit the GitHub repository at https://github.com/DivyanshuSaini2112/recony for issue tracking, discussions, and pull requests. Additional resources include external tool documentation for Nmap, Subfinder, Gobuster, ffuf, WhatWeb, Nikto, and SQLMap; security resources including OWASP Testing Guide and PTES Standard; and legal resources covering relevant computer security laws and regulations in various jurisdictions.